

CSC 4553

Fall 2026 - Day 8

Admin

- Don't forget to sign up for assignment 1 review
<https://calendar.app.google/71iRFEG6RNLqf9P29>
- Lab due Friday, 18:00
- Quiz due Monday, 18:00

Scheduling

Questions to consider

- What are the differences / tradeoffs between preemptive and non-preemptive scheduling algorithms?
- What are the metrics we can attempt to optimize scheduling for a given scenario?

Preemptive vs. non-preemptive scheduling

- **Non-Preemptive:** The scheduler only makes scheduling decisions when a process voluntarily gives up the CPU (e.g., by blocking on I/O, completing, or explicitly yielding).
- **Preemptive:** The scheduler can interrupt a running process and move it to the ready queue at any time (based on time slices, priority changes, etc.), even while it's actively executing.
- Non-preemptive is easier to implement, but
 - ... doesn't support multiprogramming very well
- Preemptive can be complicated: correctness, utilization, flexibility
 - Say a process P is preempted while in the middle of inserting an element in a data structure that another process (which is chosen by the scheduler) is going to process

Scheduling metrics

- **CPU utilization**: keep the CPU as busy as possible
- **Throughput**: # of processes that complete their execution per unit time
- **Turnaround time**: amount of time to execute a particular process (from submission to completion)
- **Waiting time**: amount of time a process spends in the ready queue
- **Response time**: amount of time it takes from when a request was submitted until the first response is produced

Scheduling scenarios

Which metric would you optimize for these situations?

- You run a data center that charges based on jobs completed
 - Max throughput: maximize the number of jobs processed per unit time
 - You run GeForce Now
 - Min response time: user experience is the main concern
 - You run Github Actions or Travis CI
 - Min turnaround time: developers need feedback quickly
 - You run a supercomputing center
 - Maximize CPU usage: this stuff is expensive, use it efficiently
- CPU utilization
 - Throughput
 - Turnaround time
 - Waiting time
 - Response time

Example: Scheduling a 911 system

You're designing a scheduler for a 911 emergency dispatch system. **Which metric matters most? Are there any metrics from the list that would be actively harmful to optimize for?**

What does the OS scheduler know about a process?

- Process characteristics that we'd like to know in order to achieve our goals
 - Processing Time (Burst Time)
 - I/O Requirements
 - Priority
 - Age
 - Dependencies
- The scheduler knows very little of the above
- In many cases we are hoping to predict the future. Easy, right?

What isn't clear?

Comments? Thoughts?

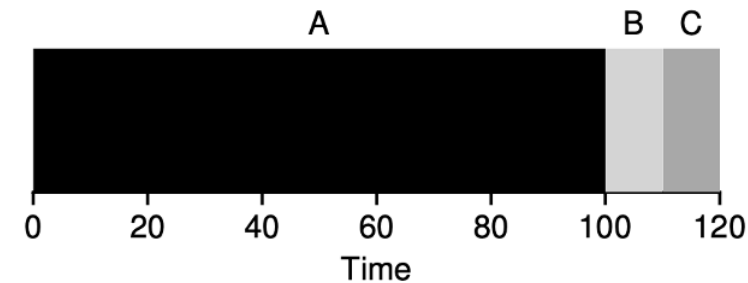
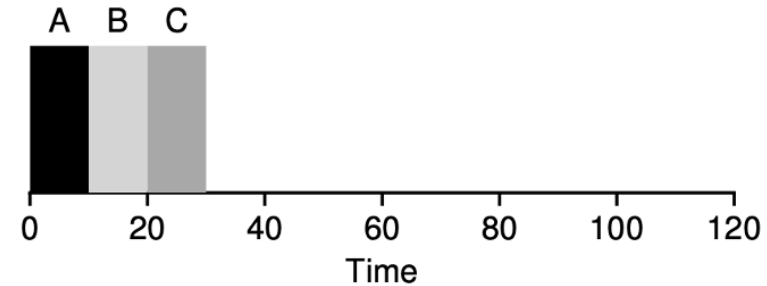
Batch scheduling algorithms

Questions to consider

- How do FCFS, SJF, and SRTF compare in terms of tradeoffs between waiting time and response time?
- Why does SJF require predicting future CPU bursts, and how can we estimate them?
- What is starvation in the context of scheduling, and how can we prevent it?

Batch scheduling: FCFS

- First-Come, First-Served (FCFS)
- **Non-preemptive**
- Pros:
 - Easy to understand, easy to implement
- Cons?
 - Waiting time & turnaround time have high variance
 - Example: 1 long CPU-bound process and many I/O bound processes
 - Convoy effect



FCFS example

- Calculate the average waiting time:
- Order: P1, P2, P3

Process	Burst Time
P1	24
P2	3
P3	3



- Waiting time P1 = 0; P2 = 24; P3 = 27
- Average waiting time: $(0 + 24 + 27)/3 = 17$
- Order P2, P3, P1



- Waiting time P1 = 6; P2 = 0; P3 = 3
- Average waiting time: $(6 + 0 + 3)/3 = 3$

Okay, FCFS can be bad

Can we think of a better algorithm to deal with the reality that jobs run for different amounts of time?

Batch scheduling: shortest job first (SJF)

- Ordered by length of the process's next CPU burst

- **Non-preemptive**

- Pros?

- Optimal average waiting time among non-preemptive algorithms

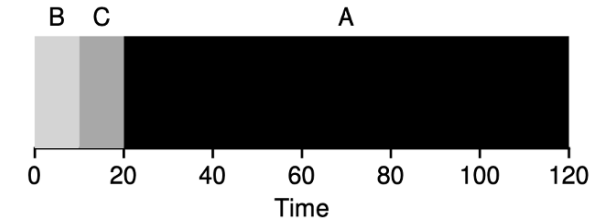
- Cons?

- Requires knowing the future (potentially okay in some situations; examples?)

- Scientific jobs, batch jobs

- Question: how do we know the length of the upcoming CPU burst?

- Ask the process? Estimate? How?



SJF: how can we predict the future?

- We can't trust a process to tell the truth
- We can only estimate, how?
 - The past (with the hope that the process will act like it has before)
- Exponential weighted moving average:

$$EWMA_t = \alpha \times x_t + (1 - \alpha) \times EWMA_{t-1}$$

- $\alpha = 0$? New measurement does not get taken into account
- $\alpha = 1$? History does not get taken into account
- α is often set to .5

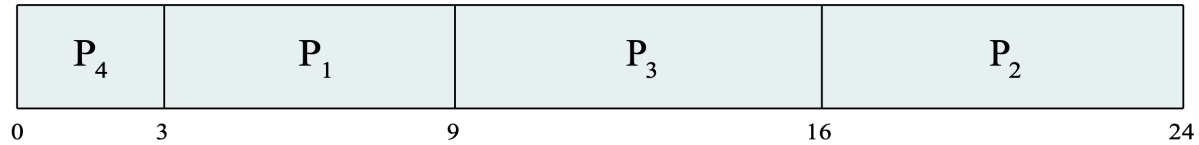
EWMA example

$$EWMA_t = \alpha \times x_t + (1 - \alpha) \times EWMA_{t-1}$$

- Suppose $\alpha = 0.5$ and a process has had burst times: 10, 6, 4, 6, ...
- $EWMA_0 = 10$
- $EWMA_1 = 0.5 \times 6 + 0.5 \times 10 = 3 + 5 = 8$
- $EWMA_2 = 0.5 \times 4 + 0.5 \times 8 = 2 + 4 = 6$
- $EWMA_3 = 0.5 \times 6 + 0.5 \times 6 = 3 + 3 = 6$
- Recent values pull the estimate toward them, but history still matters

SJF example

- Gantt:



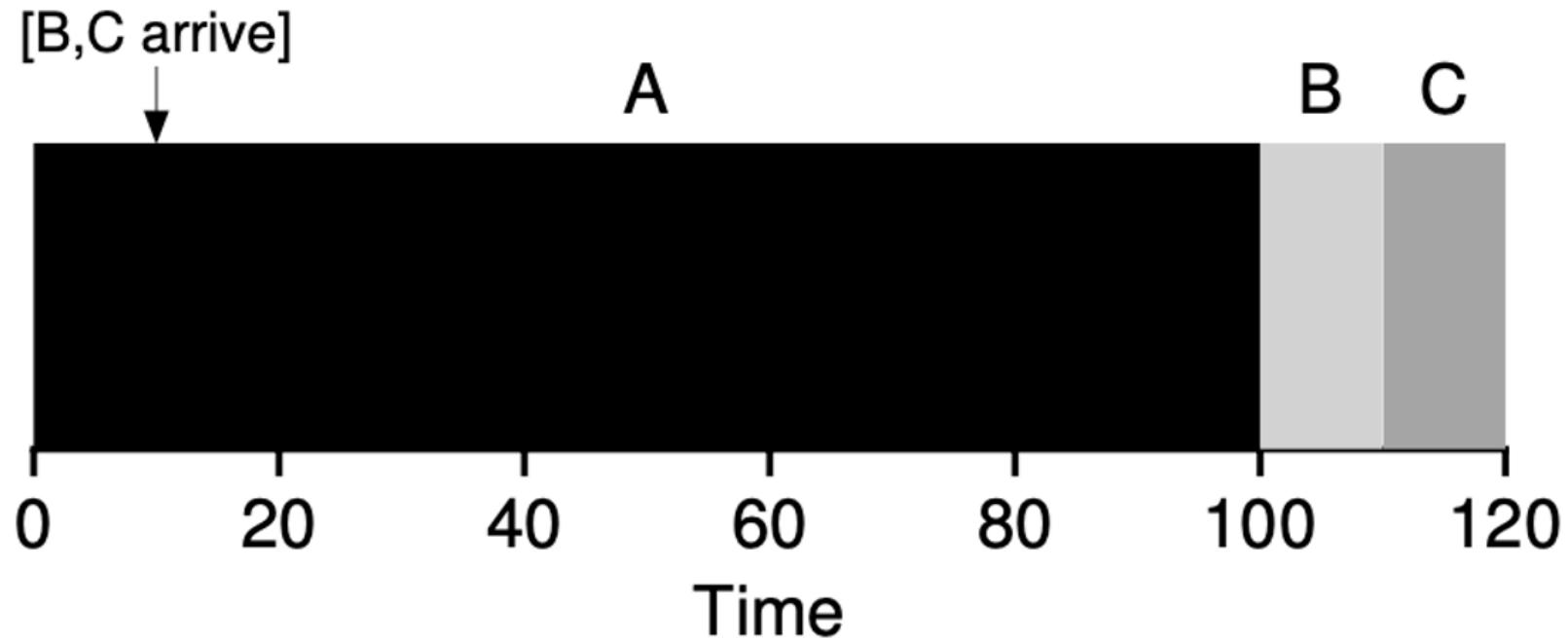
Process Burst Time

P1	6
P2	8
P3	7
P4	3

- Average waiting time: $(3 + 16 + 9 + 0)/4 = 7$
- Versus FCFS: $(0 + 6 + 14 + 21)/4 = 10.25$

Do you see any problems with SJF?

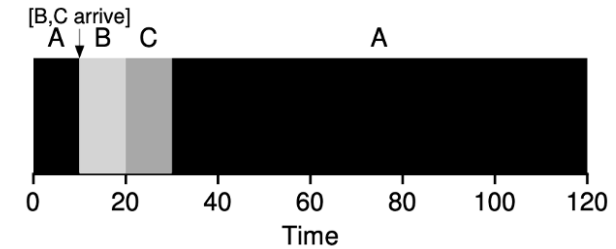
- What do we do if a short process arrives after SJF has started a long running process?



- What should we do?
 - Preempt

Shortest remaining time first (SRTF)

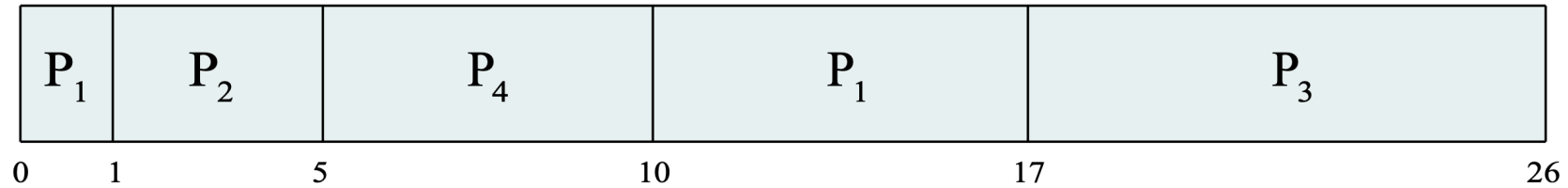
- **Preemptive** version of SJF
- Whenever a new process arrives in the ready queue, the decision on which process to schedule next is redone using the SJF algorithm
- **Question:** is SRTF more optimal than SJF for any metrics?
 - Yes, average waiting time is minimized



SRTF example

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

- Average waiting time = $[(0 + 9) + (0) + (15) + (2)]/4 = 26/4 = 6.5$



Do you see any problems with SRTF?

- Computers aren't static. What if short jobs continuously arrive?
 - Large jobs can **starve**
 - *Urban legend about IBM 7074 at MIT: when shut down in 1973, low-priority processes were found which had been submitted in 1967 and had not yet been run...*

Priority scheduling

- “Shortest” algos are examples of priority scheduling
- I.e., scheduling is based on some metric of priority
- Downsides
 - If not implemented carefully, indefinite blocking / starvation
 - How do we prevent starvation?
 - **Aging**: gradually increase the priority of processes that wait in the system for a long time

What isn't clear?

Comments? Thoughts?